

Question 1:

Use the systemctl command to check the status of the sshd service on your Linux system and write down whether it is active, inactive, or failed.

Answer:

- **Command:**

```
sudo systemctl status sshd
```

(Note: On Ubuntu/Debian systems, the service may be named ssh: sudo systemctl status ssh)

- **Result / State Explanation:**

- Look at the Active: line in the terminal output:
 - **active (running):** The SSH daemon is currently up, running in memory, and accepting remote connections.
 - **inactive (dead):** The service is stopped and not consuming resources.
 - **failed:** The service encountered a fatal configuration error or crashed upon startup.

Question 2:

Start the nginx service using systemctl, then immediately stop it. After each action, verify the status and note the changes.

Hint: Use 'sudo systemctl start nginx' and 'sudo systemctl stop nginx'.

Answer:

- **Step 1: Start the service and verify:**

- Command to start: `sudo systemctl start nginx`
- Command to check status: `sudo systemctl status nginx`
- **Observed State:** The output displays Active: active (running) in green along with the master and worker process IDs (PIDs), indicating the server is actively listening for incoming web traffic.

- **Step 2: Stop the service and verify:**

- Command to stop: `sudo systemctl stop nginx`
- Command to check status: `sudo systemctl status nginx`

- **Observed State:** The status changes to Active: inactive (dead), and the child worker processes terminate, showing that the web server has safely shut down and released port 80.

Question 3:

Enable the cron service to start automatically on boot, then reboot your system and confirm that the service is running after reboot.

Answer:

- **Step 1: Enable the cron service to start on boot:**

- Ubuntu/Debian: `sudo systemctl enable cron`
- RHEL/CentOS: `sudo systemctl enable crond`

(Output confirms the creation of a symlink in /etc/systemd/system/multi-user.target.wants/).

- **Step 2: Reboot the operating system:**

`sudo reboot`

- **Step 3: Confirm status after system boots up:**

- Ubuntu/Debian: `systemctl status cron`
- RHEL/CentOS: `systemctl status crond`

- **Verification:**

The output shows Loaded: ... enabled; on the Loaded line and Active: active (running) on the Active line, confirming the service automatically spawned during system initialization.

Question 4:

List all currently running services using systemctl and identify any service related to networking (like NetworkManager or networking). Write the current status of that service.

Answer:

- **Step 1: List all active running services:**

`systemctl list-units --type=service --state=running`

- **Step 2: Identify and check the networking service:**

- Filter the running services:

```
systemctl list-units --type=service | grep -E "NetworkManager|networking"
```

- Check the specific status (example for NetworkManager):

```
systemctl status NetworkManager.service
```

- **Current Status Output:**

The service displays Active: active (running). It indicates that the network stack is up, network interfaces are managed, and IP routing is functional.

Question 5:

Simulate a real-world scenario: Imagine you are troubleshooting why Spotify Web cannot stream music on your Linux machine. Use systemctl to check if the firewall (ufw or firewalld) service is running, and if it is, disable it temporarily. Document each command used and the result.

Constraint: Only disable the firewall if you are on a safe, non-production environment.

Answer:

- **Step 1: Check firewall service status:**

- For UFW (Ubuntu/Debian):

```
sudo systemctl status ufw
```

(Or check active status: sudo ufw status)

- For Firewalld (RHEL/CentOS/Fedora):

```
sudo systemctl status firewalld
```

- **Result:** Status indicates Active: active (running).

- **Step 2: Temporarily stop/disable the firewall service:**

- For UFW:

```
sudo systemctl stop ufw
```

(Or disable via tool: sudo ufw disable)

- For Firewalld:

```
sudo systemctl stop firewalld
```

- **Step 3: Verify the firewall is stopped:**

- Command: sudo systemctl status ufw (or sudo systemctl status firewalld)

- **Result:** Output transitions to Active: inactive (dead). With packet filtering disabled, test Spotify Web in the browser to isolate whether firewall port blocking was the root cause.
- **Step 4: Re-enable firewall after troubleshooting (Restoration step):**
 - Command: `sudo systemctl start ufw` (or `sudo systemctl start firewalld`)
 - **Result:** Status returns to Active: active (running), restoring system security.